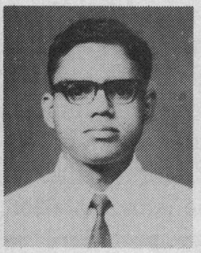
[18] S. B. Marincović and Z. Tošic, "Algorithms for minimal polarized form determination," IEEE Computer Group Repository, Paper R-74-13, Jan. 1974.



K. L. Kodandapani was born in Bangalore, India, on August 8, 1949. He received the B.E. degree in electrical engineering from the University of Mysore, Mysore, India, in 1969, and the M.E. (with distinction) and Ph.D. degrees in electrical engineering from Indian Institute of Science, Bangalore, India, in 1971 and 1974, respectively. His research interests are in the areas of switching and automata theory, logic design, fault-tolerant computing.

Rangaswamy V. Setlur was born in Bangalore, India, on April 27, 1936. He received the B.Sc. degree from the University of Mysore, Mysore, India, in 1953, the D.M.I.T. degree in electronics from the Madras Institute of Technology, Madras, India, in 1956, and the Ph.D. degree in electrical engineering from the University of Pennsylvania, Philadelphia, in 1967. During the periods 1958–1962 and 1967–1970 he was a member of the Computer Group of the Tata Institute of Fundamental Research, Bombay, India. Since 1970 he is on the faculty of the Indian Institute of Science, Bangalore, India, and is a Professor in the School of Automation. His research interests are in the areas switching theory and logic design, automata theory, mathematical logic, mathematics of computation, and systems programming.

Dr. Setlur is a member of the Computer Society of India.

Decomposition of Polygons Into Simpler Components: Feature Generation for Syntactic Pattern Recognition

HOU-YUAN F. FENG AND THEODOSIOS PAVLIDIS, MEMBER, IEEE

Abstract—**A technique for decomposition of polygons into simpler components is described and illustrated with applications in the analysis of handwritten Chinese characters and chromosomes. Polygonal approximations of such objects are obtained by methods described in the literature and then parts of their concave angles are examined recursively for separating convex or other simple shape components. Further decomposition of the latter is possible. The final result can be expressed as a labeled graph and processed further through the introduction of either fuzzy predicates or syntactic pattern recognition techniques.**

The resulting descriptions are invariant under a number of transformations and therefore there is no need for registration and normalization of the input.

Index Terms—**Chinese character description, chromosome description, feature generation, polygonal decomposition, syntactic pattern recognition.**

I. INTRODUCTION

THIS paper describes a method for decomposing plane sets of complex shape into simpler components. Such sets can be outlines of alphabetic characters, or of objects in biomedical pictures, or of objects in aerial photographs, etc. The decomposition into simpler components and the description of the set (or the object it represents) through them and their juxtaposition relation results in a labeled graph which can then be analyzed by structural or linguistic pattern recognition techniques [1]–[5]. The method is particularly appropriate for applications where the "intuitive" description of the problem expresses their class membership or characterization through such components. For example, characters of both the roman and the Chinese alphabet can be expressed in terms of strokes, and a number of recognition methods based on such decompositions are described in the literature [6]–[14]. The same is true about chromosomes [15]–[21]. However the analysis into subparts is a nontrivial problem and many of the earlier techniques have been indirect [8], [12], [19] or had considerable computational requirements.

The present method is based on approximating the boundary of the sets by polygons and then proceeding with the decomposition on the basis of convexity. A number of algorithms for these operations have been described in the literature [22]–[25]. The development and implementation of the particular algorithms for separating objects from their background and for obtaining the polygonal approximation of their boundaries is described elsewhere [25]–[28]. This software has been used for preprocessing all the data used in the examples of this paper.

Manuscript received April 16, 1974; revised October 28, 1974. This work was supported by the NSF under Grant GK-36180.

H.-Y. F. Feng was with the Department of Electrical Engineering, Princeton University, Princeton, N. J. He is now with the Pattern Analysis and Recognition Corporation, Rome, N. Y.

T. Pavlidis is with the Department of Electrical Engineering, Princeton University, Princeton, N. J.

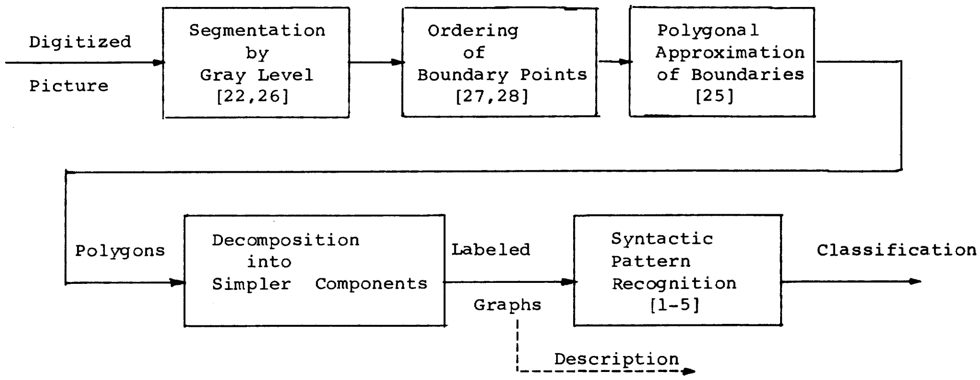


Fig. 1. Steps of a process for shape recognition. This paper describes algorithms for the fourth block.

These as well as the decomposition algorithms are quite general and need only the specification of a small number of parameters. This is illustrated by the use of two somewhat unrelated objects in our examples: Handwritten Chinese characters and chromosomes. This is useful for pattern recognition applications since one will not have to redesign the complex feature extraction software as long as his problem falls in the rather broad class where decomposition is the proper method. Fig. 1 outlines a complete pattern recognition system. This paper deals with the fourth block.

A method of polygonal decomposition had been proposed earlier by one of the authors [4], [29], [30]. However its computational requirements were rather high and the new scheme offers a compromise by giving up some of the desirable features of that decomposition in order to reduce the amount of computation required. The older scheme was theoretically more elegant and produced decompositions which were independent of the indexing of the corners of the input polygon. The present scheme in some cases produces results which depend on the indexing (see Section III). On the other hand it does not have the restriction that the decomposition be made only into convex sets. This was an essential requirement of the former. All the computations now deal with the vertices of the approximating polygon and even though they may seem complex, the small size of the data set involved results in very short time. The most time-consuming step is the polygonal approximation of the boundaries, but this is not overly expensive either. A more detailed discussion of the computational requirements is given later in the paper.

The method treats polygons without holes (simply connected) differently than those with holes. The first case is discussed in the next section and the second in Section V.

II. DECOMPOSITION OF SIMPLY CONNECTED POLYGONS AT CONCAVE ANGLES

We assume that we are given a sequence

$$VS(P_n) = (I,J)_1(I,J)_2\cdots(I,J)_n$$

which contains n corners prearranged clockwise or counterclockwise around a simply connected polygon P_n. The location of ith corner C_i in the discrete space domain is denoted as $(I_i,J_i) = (I,J)_i$.

$A(C) = \angle C'CC''$, where C',C,C'' are consecutive corners, denotes the *spanning angle* at a corner C with respect to P_n.

A corner C is an a-corner if $A(C) < 180°$, C is a b-corner if $A(C) > 180°$. By the definition of a "corner", $A(C) \neq 180°$.

The *angular characteristic* of P_n is defined as a string

$$AC(P_n) = x_1x_2\cdots x_n \in \{a,b\}^+$$

derived from $VS(P_n)$ such that $x_i = a$ iff C_i is an a-corner and $x_i = b$ iff C_i is a b-corner.

We know from elementary plane geometry that at least three corners in P_n must be a-corners. Thus we assume in the following discussions that $VS(P_n)$ has been permuted so that $x_1 = a$ or $(I,J)_1$ is an a-corner.

For convenience, an a-string stands for a string of a's and a b-string stands for a string of b's. In the conventions of language theory, an a-string is denoted as a^+ [31]. Also, $a^* = \epsilon \cup a^+$ where ϵ represents the null string.

Our decomposition scheme consists of more than one step with the major step described in this section. This is a recursive procedure and the term "nondecomposable element" will refer to a subset which is of "simple" form and will not be processed any more at this step. It may well be analyzed further at subsequent steps. The basic idea of decomposition is explained in the following paragraph.

Given a simply connected polygon, we want to find a way to break it into nondecomposable elements. If a polygon contains such an element attached to the rest at two corners C,C', then we will detach it at the *baseline* CC' and generate an adjacency relation between it and the main part. Since one of the resultant subpolygons may still be decomposable (the "central" or the "main" part), we are obliged to decompose it recursively until all the subparts are nondecomposable.

Fig. 2 illustrates how this idea works on an English letter H. Fig. 2(a) is the input, Fig. 2(b) and (c) are intermediate results with Fig. 2(c) the nucleus of the decom-

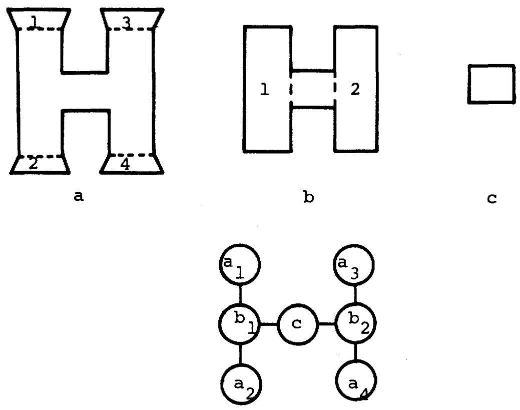


Fig. 2. Illustration of the decomposition of the roman letter H.

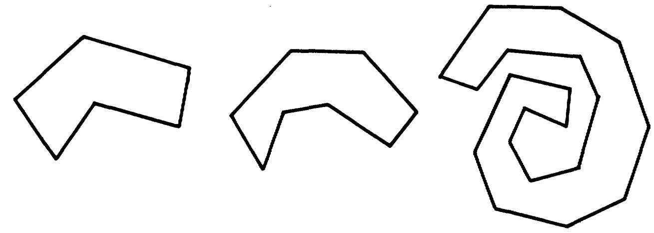


Fig. 3. Examples of spirals which are considered as primitive sets for the first decomposition. Allowing them to be nonconvex could be useful in the analysis of cursive script.

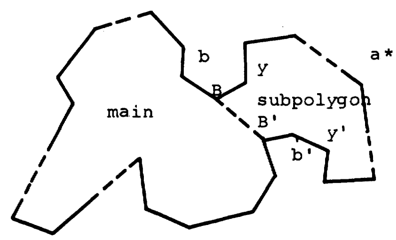


Fig. 4. The "main" and the decomposable subpart delimited by the baseline bb'.

position. The dotted lines are baselines. Fig. 2(d) is the adjacency relation among these subparts. Note that the adjacency graph is generated recursively, thus grammatical rules can be easily imposed for further descriptions.

It is obvious that concave corners are good candidates for drawing the dividing line seperating a component from the rest of the polygon. Furthermore, such corners must be separated by at least one convex one (Fig. 2). Thus we proceed to examine pairs of concave corners and check if the line joining them performs a meaningful separation. Proposition 1 below gives an estimate of the effort involved in such a decomposition.

Definition: A simple polygon P_n is *decomposable* if $AC(P_n)$ contains at least two b-strings.

This definition implies that if a polygon is convex or "spiral" (including an L, a U etc., Fig. 3) then it is nondecomposable. Such "spirals" can be decomposed in turn by a technique to be discussed later.

Proposition 1: At most four angles have to be checked to determine the decomposability of a subpolygon if its baseline is delimited at two b-corners in successive b-strings (i.e., with only one a-string between them in, say, clockwise direction).

Proof: Let the b-corners be B,B', and the subpolygon to be tested for decomposability be represented by the angular characteristics

$$a^*yx_bx_{b'}y'a^*, y, y' \in b^* \quad \text{(Fig. 4)},$$

since we assumed B,B' to be in successive b-strings.

Case 1—$x_b = x_{b'} = b$: Check the spanning angles at B,B' with respect to the polygon to be decomposed.

Case 1.1: If they both are greater than the original ones, then it is an illegal decomposition [Fig. 5(a)].

Case 1.2: If both are smaller, then we have $a^+b^+a^*$ representing the subpolygon, which is nondecomposable by definition.

Case 2—$x_b = x_{b'} = a$:

a) $y = y' \in b^+$: "T"-type, decomposable [Fig. 5(b)].

b) $(y \in b^+, y' = \emptyset)$ or $(y' \in b^+, y = \emptyset)$: we have $a^+b^+a^*$ in general, "spiral" type, nondecomposable by this step.

c) $y = y' = \emptyset$: convex.

Case 3—$x_b = a, x_{b'} = b$:

a) $y = \emptyset$: we have $a^+b^+a^*$, again a spiral type.

b) $y \in b^+$: "T"-type [Fig. 5(c)].

Case 4—$x_b = b, x_{b'} = a$: This is equivalent to Case 3.□

By the definition of a baseline, we see that if two b-corners are in the same b-string, they cannot form a baseline. Moreover, a global line-intersection check must be made before we conclude that a given pair of b-corners forms a legal baseline (Fig. 6). The algorithm described in the next section incorporates the above ideas.

III. THE DECOMPOSITION ALGORITHM

The following is the main decomposition algorithm.

Algorithm 2-way: Decomposition of Simply Connected Polygons at Concave Angles

Input: Ordered list of corners of a simply connected polygon $P_n, VS(P_n)$.

Output: Description of nondecomposable subpolygons, and their adjacency relations.

Procedure

Step 1: Find the angular characteristics of $P_n, AC(P_n) = x_1 \cdots x_n$.

Step 2: If $x_1 = b$, rearrange cyclically $VS(P_n)$ so that $x_1 = a$.

Step 3: If $AC(P_n) \in a^+b^*a^*$ go to Step 7.

Step 4: Let m be the number of b-strings in $AC(P_n)$. Define $b\text{-}S_0 = b\text{-}S_m$. Set update $(i) = 1$, $i = 0$ to m.

Step 5: For $i = 1$ to m do block 51.

51.1. Begin block 51.

51.2. For j = update $(i-1)$ to $|b\text{-}S_{i-1}|$ do block 52.

52.1. Begin block 52.

52.2. Let B be the jth corner of $b\text{-}S_{i-1}$.

52.3. For k = update (i) to $|b\text{-}S_i|$ do block 53.

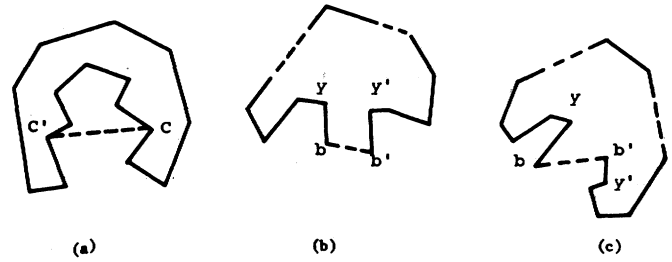


Fig. 5. (a) An illegal decomposition even though the line CC' does not intersect any boundary segment. (b) and (c) Two different types of T's discussed in Proposition 1.

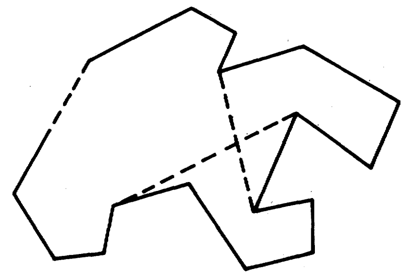


Fig. 7. A "crossed" decomposition which may happen if the b-strings are not updated in the Algorithm 2-way.

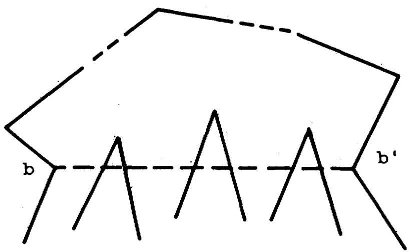


Fig. 6. A candidate for a separating line (bb') must be checked for intersections with other boundaries.

53.1. Begin block 53.
53.2. Let B' be the kth corner in b-S_i.
53.3. If the linear segment BB' intersects another side of the polygon (as determined by subroutine INTERSECT) then go to Step 53.5.
53.4. If the linear segment BB' is a legal baseline (as determined by subroutine BASELINE) then go to Step 51.3.
53.5. End block 53.
52.4. End block 52.
51.3. (The linear segment BB' separates a component.) Update the description of the string b-S_i by setting update $(i) = k$ and if $i = 1$ set $| b\text{-}S_m | = j$.
51.4. Set aside the description of the resulting subpolygon and its adjacency relation with the polygon from which was removed. Place in a stack S a pointer pointing to it.
51.5. End block 51.

Step 6: If S is empty go to Step 7. Else remove its last element, consider as "main" polygon the subpolygon pointed by it and go to Step 1.

Step 7: Output the resultant descriptions and relations and stop. *End of Procedure.*

We have to update the b-strings in Step 51.3 not only to save execution time, but mainly to prevent the "crossed" decompositions shown in Fig. 7.

Subroutine BASELINE incorporates the checks listed in the proof of Proposition 1. It returns .true. if it is a legal baseline. Subroutine INTERSECT returns .true. if the line concerned intersects any line segments of the polygon. Obviously, each execution of INTERSECT requires time of order n, thus the time of Algorithm 2-way needs time between $m \cdot n$ and $m \cdot n \cdot \max | b\text{-}S |^2$ for each round of decomposition, where n is the number of corners, m is the number of b-strings, and $| b\text{-}S |$ is the length of a b-string.

Proposition 2: Algorithm 2-way terminates.

Proof: We will use the following two lemmas from elementary geometry.

Lemma 2.1: The number of intersections of a line and a polygon must be even, if no intersections appear at the corners of the polygon.

Lemma 2.2: If the number of intersections of a line and a simply connected polygon exceeds two, then the polygon has one or more b-corners on at least one side of the line.

Now suppose that there are m b-strings in $AC(P_n)$.

Case 1: $m \leq 1$, terminate by Step 1.

Case 2: $m = 2$; Suppose there are m' b-corners in total, $m' \geq 2$.

Case 2.1: $m' = 2$, say b,b', then by Lemmas 1 and 2, INTERSECT $(b,b') =$ .false. and by Proposition 1, BASELINE $(b,b') =$ .true.

Case 2.2—$m' > 2$: Let the string be $a^+b_1\cdots b_k a^+ b_1' \cdots b_{k'}' a^*, k + k' = m'$. Set $b \leftarrow b_k, b' \leftarrow b'_1$.

Step 2.1: If INTERSECT $(b,b') =$ .false. and Case 1.1 of Proposition 1 is also .false., then go to Step 2.2. Otherwise, by Lemma 2, there exists a b-corner b'' in the polygon $b'ba^+b^*$ [Fig. 8(a)].

Since $m = 2$, $b'' \in \{b_1', \cdots, b_{k'}',\}$ or $b'' \in \{b_1, \cdots, b_k\}$. If $b'' \in \{b_1, \cdots, b_k\}$ then set $b \leftarrow b''$, otherwise set $b' \leftarrow b''$ and go to Step 2.1.

This process must terminate because m' is finite. Also, there must be a pair of b,b' that INTERSECT (b,b') is false because of Lemma 2.

Step 2.2: Now we have two subpolygons $a^+b^*a^*b^*$. By Step 2.1, there is no line intersection in it. Also, at least one of them has the number of b-corners less than that of the original one. Thus applying Cases 1 and 2 again, we can be assured of its termination.

Case 3—$m > 2$: Suppose we have m' b-corners in total, $m' \geq m$. Let there be a substring $b_1 \cdots b_k a^+ b_1' \cdots b_{k'}'$, where $b\text{-}S_{i-1} = b_1 \cdots b_k$, $b\text{-}S_i \triangleq b_1' \cdots b_{k'}'$. Set $b \leftarrow b_k$, $b' \leftarrow b_1'$.

Step 3.1: Check INTERSECT (b,b'). If .false. and Case 1.1 of Proposition 1 is also .false., then go to Step 3.2, otherwise get the b-corner b'' between the two concave angles [Fig. 8(b)] or inside the polygon $b'ba^+b^*$ [Fig. 8(c)].

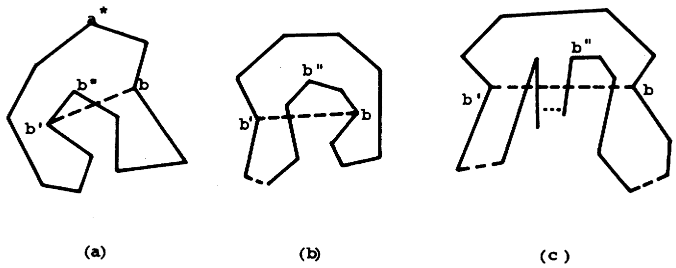


Fig. 8. Possible locations of b-corners between a test baseline bb'.

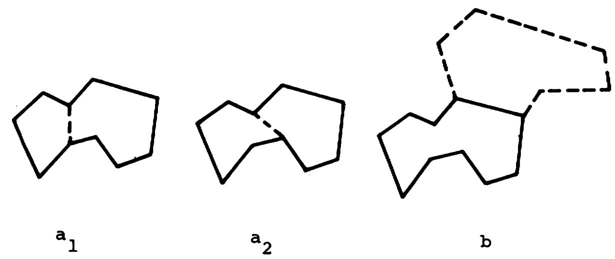


Fig. 9. An example illustrating the orientation dependence of the Algorithm 2-way.

If $b'' \in \{b_1', \cdots, b_{k'}'\}$, then set $b' \leftarrow b''$, otherwise, set $b \leftarrow b''$ and go to Step 3.1.

This process must stop by the same reason as in Step 2.1, Case 2.

Step 3.2: Now we get a subpolygon $b'b(a^+b^+)^*a^*$.

Case 3.1: If it is nondecomposable, then by the definition, b,b' belong to successive b-strings. Otherwise,

Case 3.2: Apply Cases 1, 2, or 3 and process this smaller polygon.

In both cases, we are assured of its termination because

1) the locations of corners in the subpolygon are not changed; and

2) if there exists a decomposable subpart in Step 3.2, then it is also a decomposable subpart of the original one.

Thus we see each polygon that contains more than two b-strings will be decomposed. That is, each execution of Step 5 in the algorithm will decrease the total number of b-string. Therefore, Algorithm 2-way terminates. □

Although Algorithm 2-way gives an answer to the decomposition scheme, its result depends on the ordering of the polygon vertices. Indeed:

Proposition 3: Algorithm 2-way is $VS(P_n)$ dependent.

Proof: Fig. 9(a) shows two possible decompositions based on the orientation of $VS(P_n)$. Since the particular polygon may be an intermediate result of the processing, the dependence is apparent [Fig. 9(b)]. □

Given a fixed oriented $VS(P_n)$, Algorithm 2-way produces results which are independent of the initial corner because the pairs of b-corners are picked up successively and tested in fixed order. Thus usually the resulting shapes are close to each other. It is possible to derive algorithms which give the same results regardless of the orientation of $VS(P_n)$ but this can be done only at a substantial increase of computational costs or a restriction of the class of input [32].

IV. FURTHER DECOMPOSITION AND SHAPE DESCRIPTION

If desirable, components of a spiral form can be easily decomposed into convex subsets by successive examination of their concave corners.

Algorithm S: Decomposition of Spirals into Convex Subsets

Step 1: Let the n corners of a spiral region be arranged as $C_1C_2\cdots C_m\cdots C_n$ in which the substring $C_2\cdots C_m$ are b-corners and C_1 is an a-corner (Fig. 10).

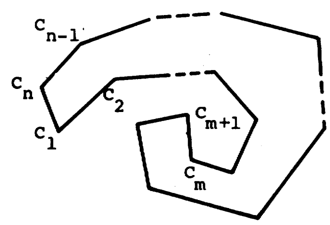


Fig. 10. A spiral with its corners labeled as in the Algorithm S.

Step 2: $k1 \leftarrow n$, $i1 \leftarrow 2$, location $\leftarrow k1$.

Step 3: For $k = k1$ step by -1 until $m + 1$ do the following:

location $\leftarrow k$.

if $\angle C_{i1-1}C_{i1}C_k$ is concave, the location $\leftarrow k + 1$ and go to Step 4.

End of Step 3.

Step 4: $k1 \leftarrow$ location. Decompose the spiral at (C_{k1},C_{i1}).

Step 5: If the angle $\angle C_{k1}C_{i1}C_{i1+1}$ is concave then $i1 \leftarrow i1$, otherwise, $i1 \leftarrow i1 + 1$.

Step 6: If $i1 > m$ then stop, otherwise go to Step 2.

End of the Procedure.

It is obvious that the above procedure works. We can actually put additional criteria such as the minimal distance between two corners to make the procedure more suitable to the problem. Convex sets, or even spirals, can be described in terms of relatively few primitive shapes and this subject is discussed in the literature [33]–[36]. For example, it is easy to derive for them an elongation number as the ratio of the maximum over the minimum width [37].

V. DECOMPOSITION OF POLYGONS WITH HOLES

The picture segmentation algorithms developed yield the boundary of regions as a sequence of polygons, approximating each connected boundary segment [28]. In general, we can design algorithms for such decomposition by any of the following three approaches.

Approach 1

1) For each polygon which is the boundary of a hole, we find a vertex which is closest to one of the external boundary. Then each vertex is split and a pair of "pseudosides" is inserted by joining them as in Fig. 11. This will yield a simply connected polygon.

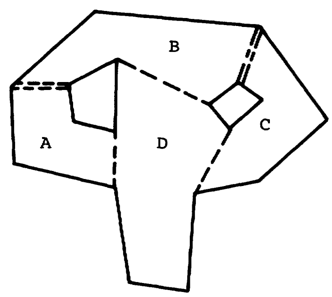


Fig. 11. Decomposition of a polygon with holes by Approach 1. "= = = =" are the pseudoboundaries.

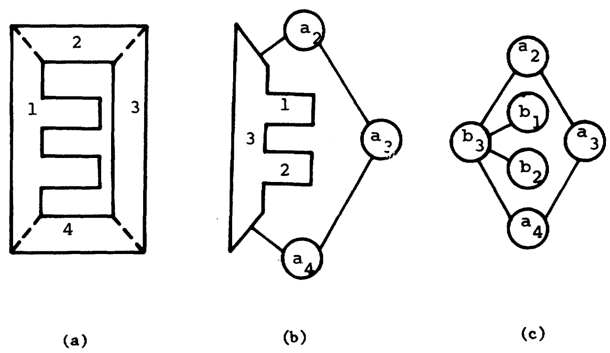


Fig. 12. Decomposition of a polygon with holes by Approach 2.

2) Apply the algorithm of Section II.

3) Establish the adjacency relations among all pairs of components which share a pair of pseudosides (*A-B* and *B-C* in Fig. 11).

It is easy to show that the merging of a components across pseudosides will result in convex sets.

Approach 2: This is a Modification of the First Approach

1) Separate the region into simply connected subregions by matching pairs of corners on different closed boundaries, if their distance is below a threshold value. Generate adjacency relation among the subregions.

2) Decompose each subpart by using the algorithm of Section II.

3) Generate the overall adjacency relation as in the first approach.

This approach works when the picture is primarily made of strokes because then such pairs of corners are easy to find. For example, a Chinese character "the eye" can be decomposed as shown in Fig. 12.

Approach 3

1) Describe the region by the decomposition method discussed earlier as though there were no holes inside it.

2) Describe the holes as independent simply connected regions.

3) Generate relations among the holes such as the relative position of each pair of them.

This approach can be applied on problems where the relative position among corners does not play a crucial part or there are so many holes in the region that the first two approaches become impractical. For instance, this can be the case with blood cells, human faces or scenes of objects which hide each other. The relational graphs generated can be further processed by relational grammars like those suggested in [38].

It should be emphasized that in a number of recognition problems a complete decomposition will not be necessary. For example, the result of the segmentation which describes regions in terms of linearly ordered boundaries [28] allows immediate discrimination on the basis of topological description since it readily yields the order of connectivity [39]. It may be necessary to obtain polygonal approximations only for some of the boundaries and do complete decompositions only for some parts, especially if the figure under consideration is not connected. Additional discussion on this point can be found elsewhere [32].

VI. EXAMPLES OF IMPLEMENTATION

The algorithm of Section III has been implemented in Fortran on an IBM 360/91 computer as the fourth block of the series of processing programs shown in Fig. 1. A number of pictures of chromosomes and handwritten Chinese characters were digitized by a system consisting of a closed circuit TV camera connected to a minicomputer (HP2116) via a bandwidth compressor (by Colorado Video Associates) and an A/D converter. These were stored on magnetic tape and then processed on the IBM 360/91. The standard size used was 64 $\times$ 64 with 7 bits per pixel. Figs. 13(a)–19(a) show their printouts through the use of overprinting. Figs. 13(b)–19(b) show the regions obtained by a thresholding operation (because of the high

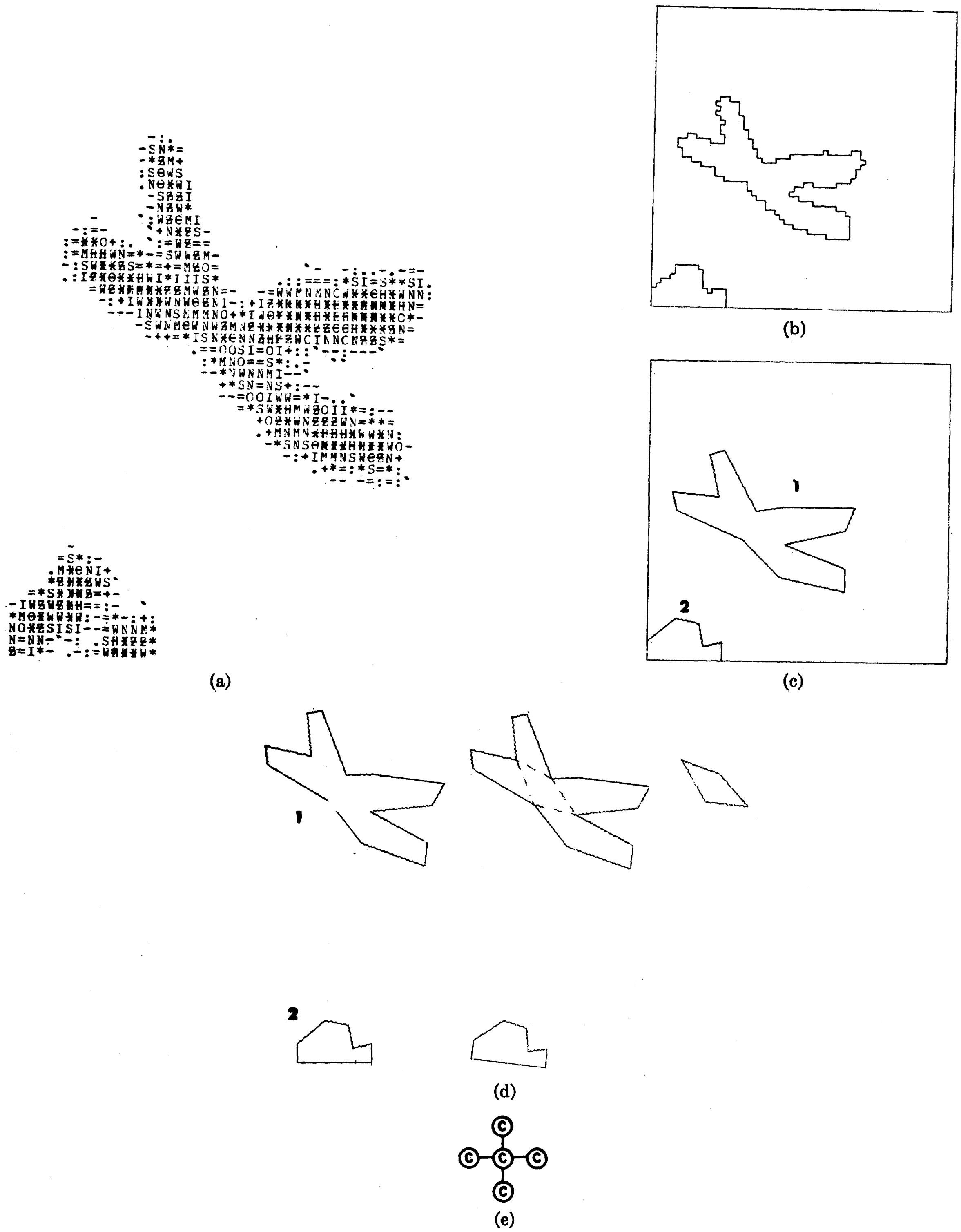


Fig. 13. A picture containing a single chromosome and a small part of another. (a) Original. (b) Contours. (c) Polygonal approximation. (d) Decomposition (baselines are shown as dotted lines). (e) A labeled graph summarizing this description. The relative location of the nodes on the plane is important. Additional labeling, nodes and branches can be introduced in the manner described in [4].

contrast there was no need for sophisticated segmentation techniques). Figs. 13(c)–19(c) show the polygonal approximations of the boundaries. Figs. 13(d)–19(d) are CALCOMP plotter outputs illustrating the decomposition by the method of this paper. Figs. 13(e)–19(e) are labeled graphs encoding the decomposition. The primitives are convex sets (C), T-type sets (T) or spirals (S). These include L-shape and U-shape objects. Fig. 20 shows the computer printout for the analysis of Fig. 19.

It is difficult to obtain an accurate estimate for the

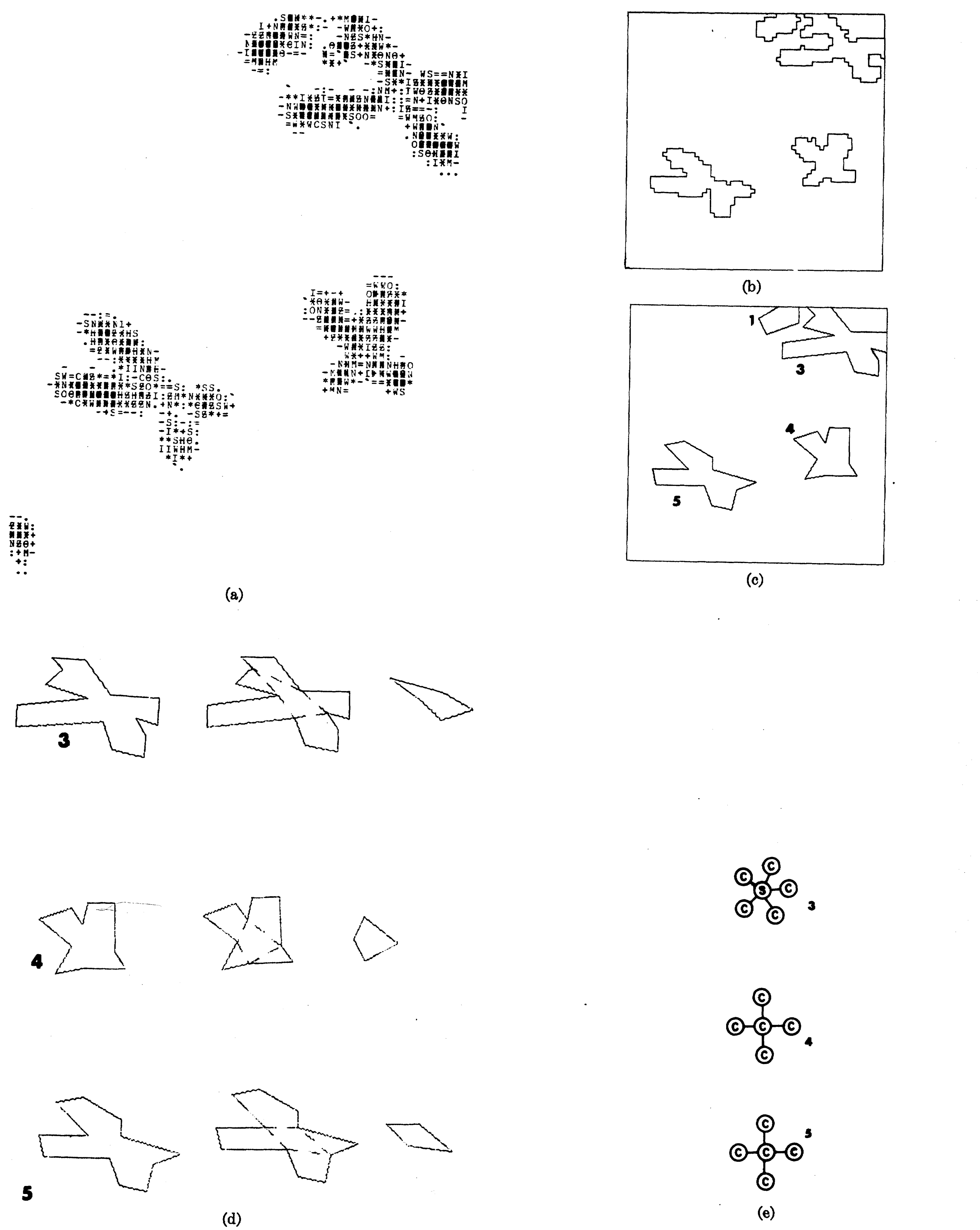


Fig. 14. Part of a picture containing a group of chromosomes. Decomposition was performed on three of them indicated by the numbers on (c) and (f).

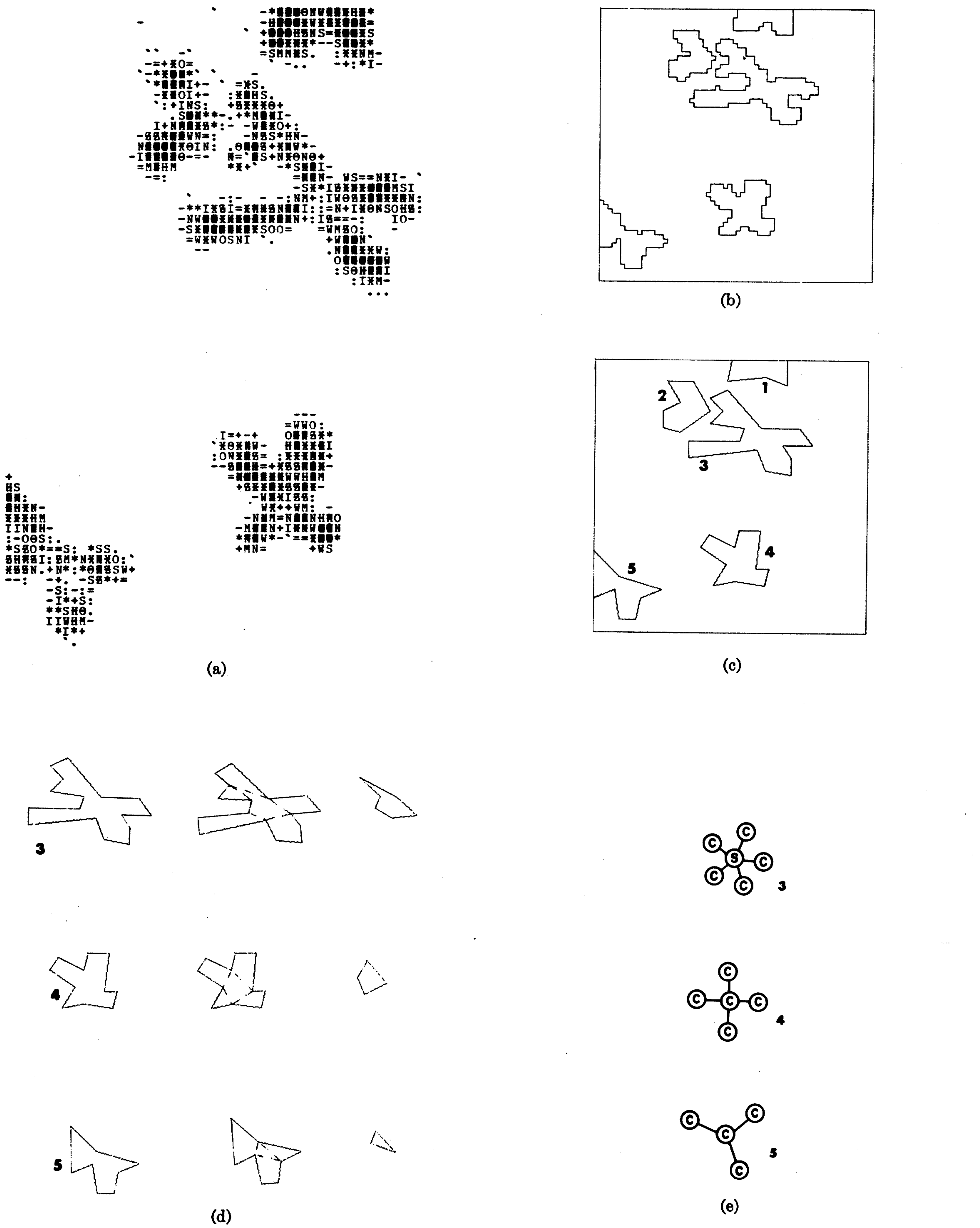
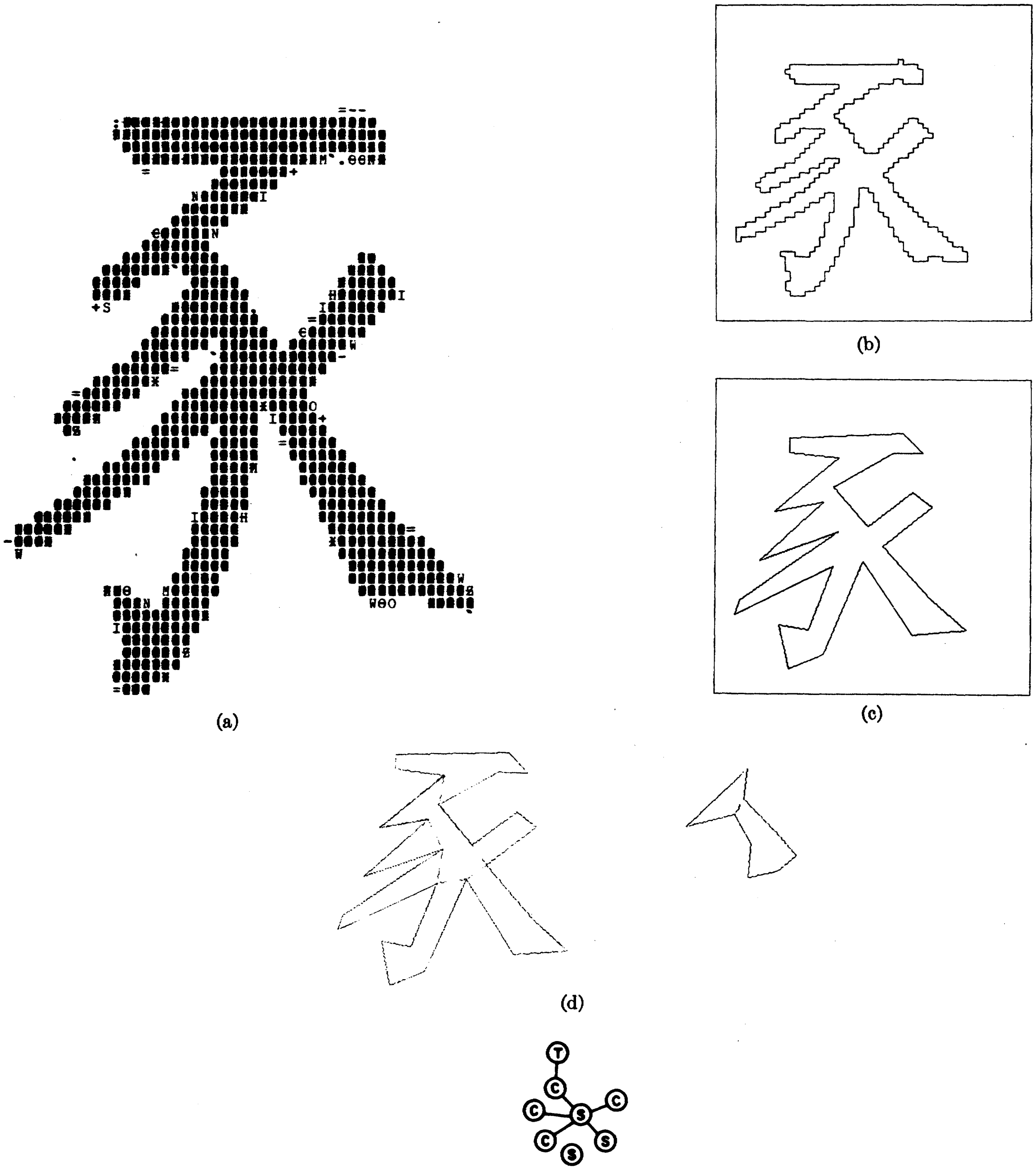


Fig. 15. Another part of the picture used for Fig. 14. Some chromosomes appear in both and the repeatability of the decompositions illustrate the robustness of the method. Even when parts are missing (no. 5) the two forms are compatible.



(a) (b) (c) (d) (e)

Fig. 16. Analysis of the Chinese character for Pig.

processing time because each of the blocks of Fig. 1 communicates with the others through a scratch disk and therefore I/O time is substantial.[1] Table I lists the relevant times. The boundary approximation by polygons is the most time-consuming part since it uses a least integral square fit with variable vertices [25]. Once the polygons are found the decomposition is extremely fast. It takes less than $\frac{2}{3}$ of the thresholding operation!

A rough idea of the eventual time requirements of the method can be given by using a speed up factor of 5:1 for a main core assembly language programs over those in Fortran or PL/I[2] and a factor of 10:1 for using some simple special hardware (e.g., shift registers for checking binary array matching) [40]. This yields about 12 Chinese characters per second or 16 chromosome pictures per second. Harmon [9] gives as an optimal speed for Chinese character recognition 3400 per minute or 56 per second.

[1] There are two reasons for this organization. One is flexibility: different blocks involve software in different languages, written by different people. The other is the increase in priority in the computer center queue for jobs with small maincore requirements, even if their execution time is longer.

[2] This is particularly true for picture processing programs where the evaluation of indices is a major part of the computation.

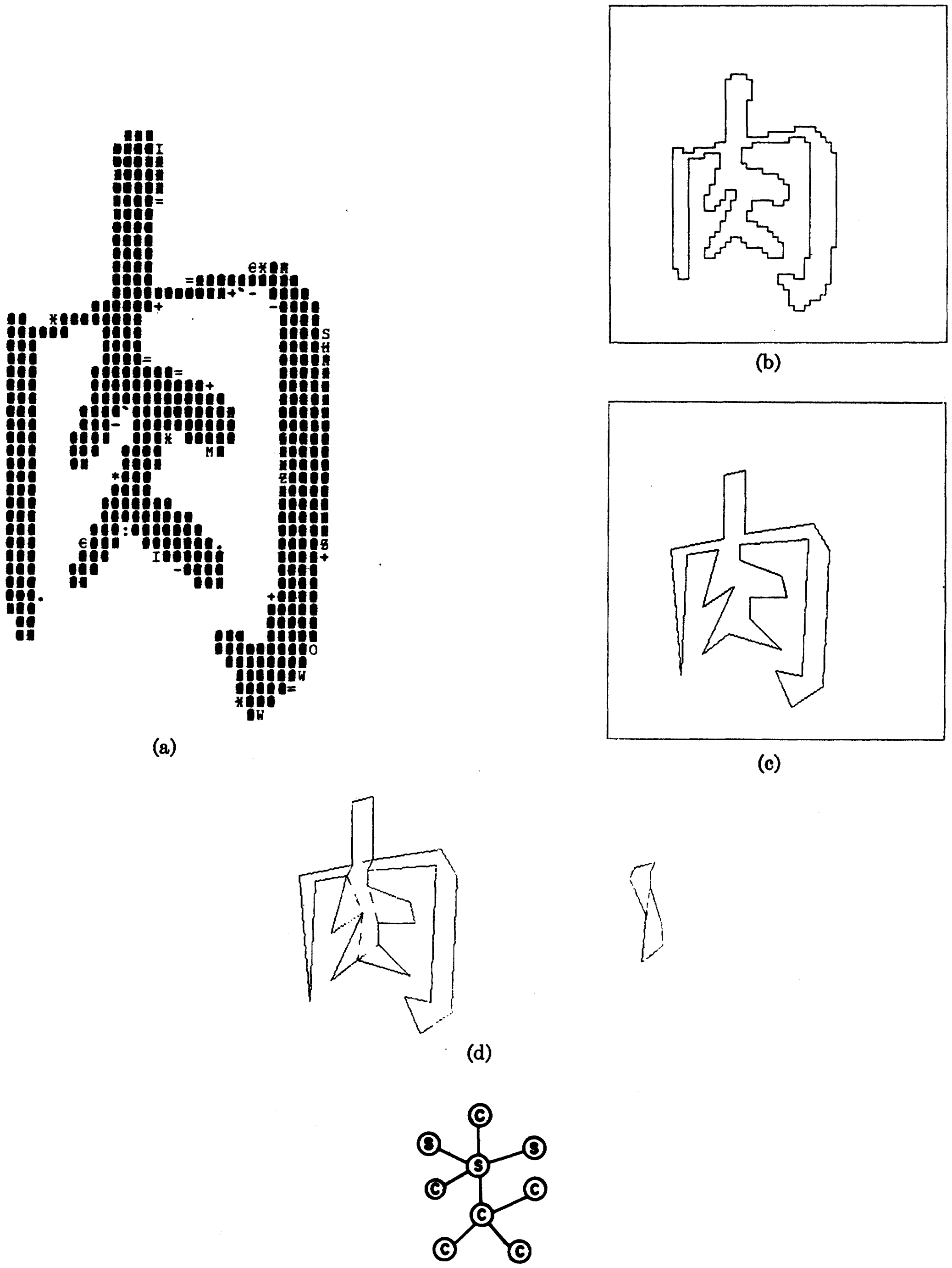


Fig. 17. Analysis of the Chinese character for Meat.

An additional speed up factor of about 5:1 would be needed to reach that speed. This can come in many ways. Reducing the quantization grid from 64×64 to 32×32, using a simpler method for polygonal approximation, using parallel processing or special complex hardware (e.g., for the line fitting subroutine). The last two ways will increase the cost while the first two will reduce it but they may also cause a deterioration in accuracy. Thus in spite of its apparent complexity the method is competitive for on-line implementation.

VII. DISCUSSION

The decompositions obtained by the method described have many desirable features.

1) They are translation and rotation invariant[3] and insensitive to registration as illustrated by Figs. 14 and 15. Rotation invariance can be controlled through the description of the juxtaposition relations in the final graph. The

[3] At least within rotations which do not affect the indexing of the input.

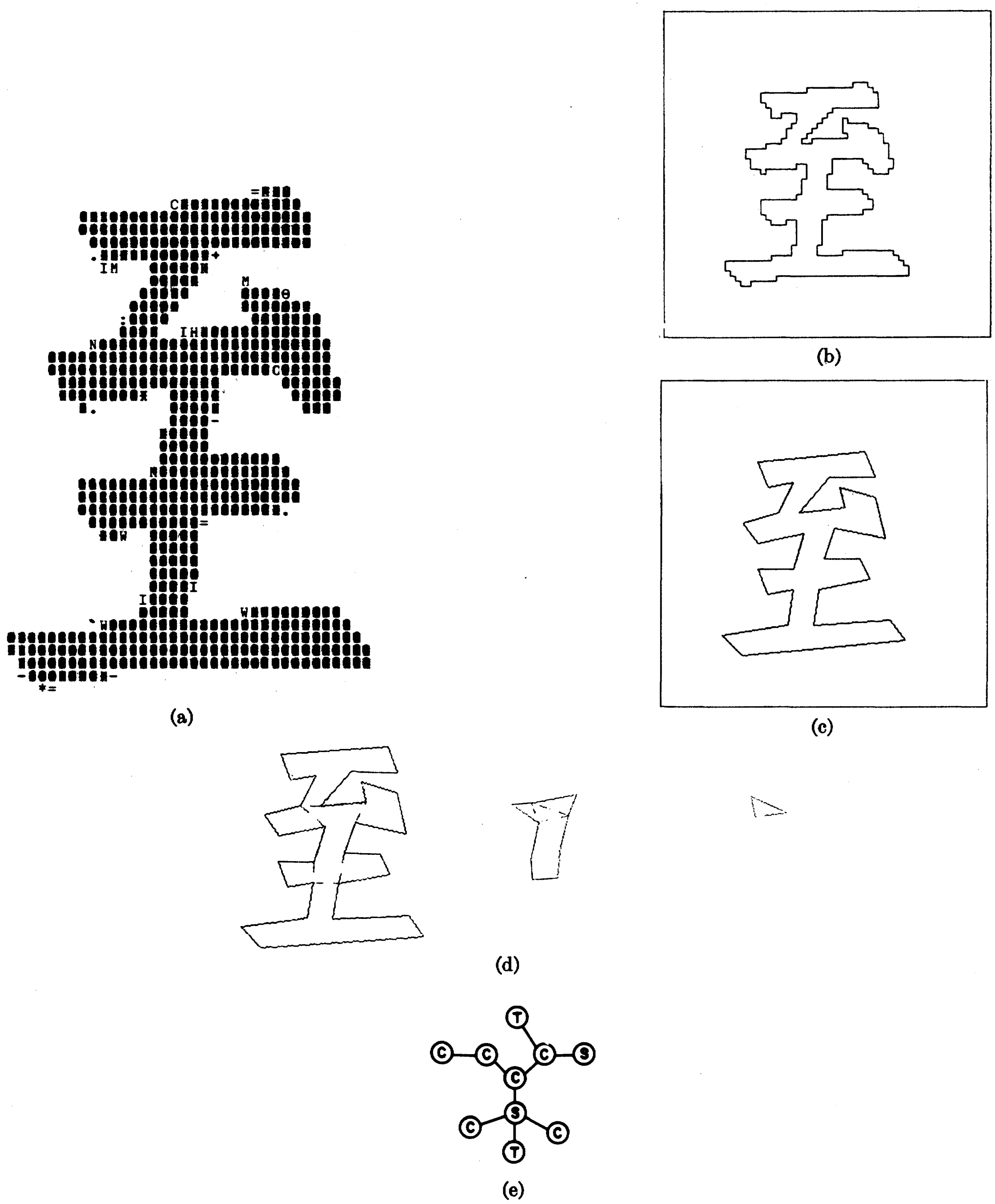


Fig. 18. Analysis of the Chinese character for the verb Reach.

insensitivity to registration is important for practical applications, e.g., optical page readers, mail sorters, biomedical picture processors, etc.

2) To a large extent, they are size invariant. Problems may occur only for very large variations and in particular when the dimensions of an object are reduced so that they are of the same order as the error tolerance for the polygonal approximation.

3) They are relatively fast, especially when compared with algorithms which obtain a detailed stroke decomposition. (They can be slower than indirect schemes, but then the latter need precise registration and normalization.) In particular their computational requirements are determined by the effort for the polygonal approximation. The time required for them is proportional to the length of the boundaries rather than the picture area (e.g., see [41]).

4) The resulting descriptions are "anthropomorphic" as illustrated by Figs. 13(f)–19(f) and therefore readily available for feature extraction.

Their use in pattern recognition can take basically one of two forms.

a) Define logical predicates through them and then use those as components for a feature vector or input to a

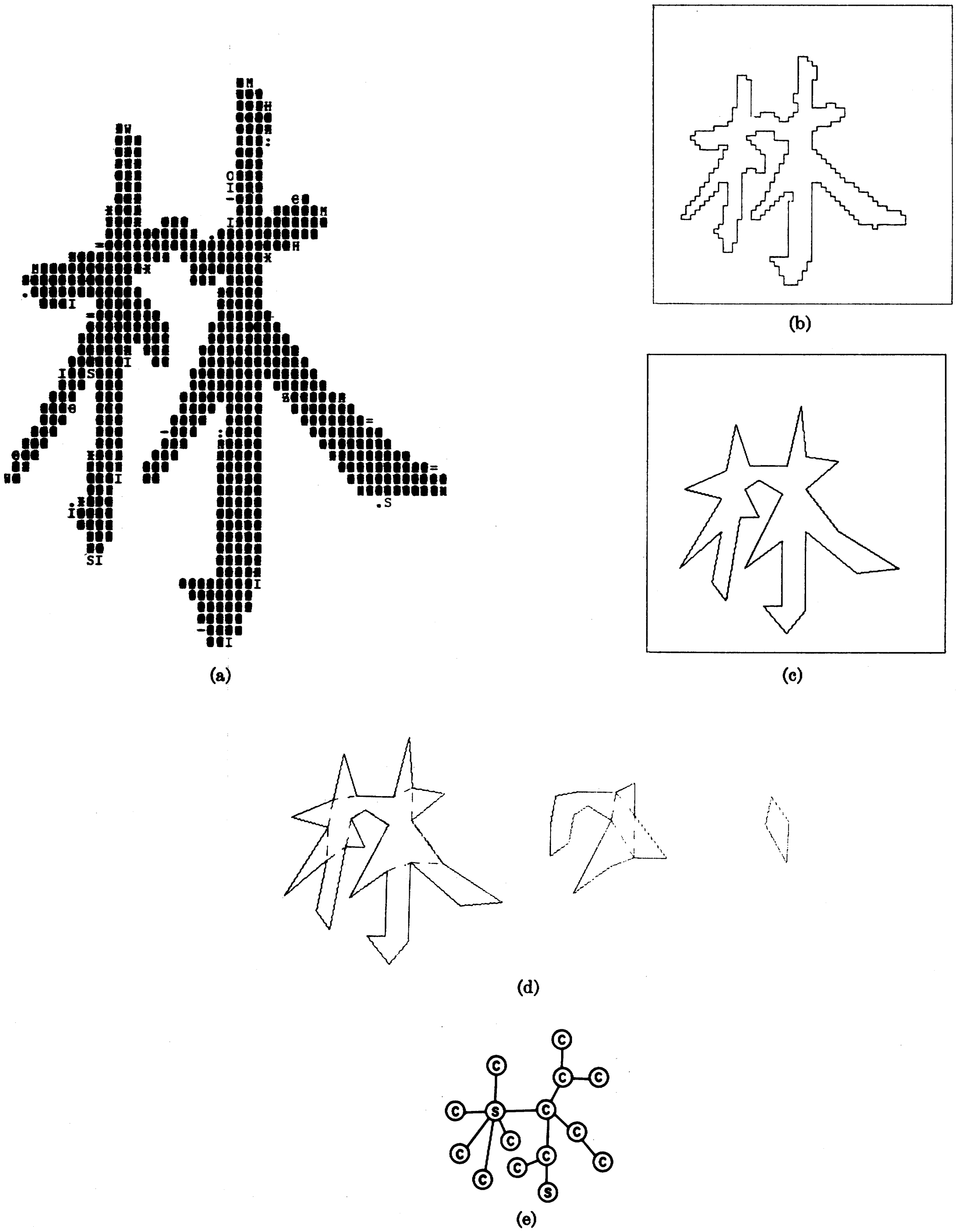


Fig. 19. Analysis of the Chinese character for Forest.

truth function. If standard logical predicates are used then a binary feature vector will result. A more attractive approach is to use fuzzy predicates [42], [43] which will result in a real feature vector. If e is the elongation number of a convex component (Section IV) then the presence of a "stroke" can be indicated by a fuzzy variable

$$s = 1 - e^{-1}.$$

For chromosomes the sum of these variables for each component could be used as the main feature.

b) For more complex patterns, graph or tree grammars and related schemes could be used [44]–[48]. Again some of the rules could be made fuzzy to avoid problems associated with overcommitment because of early segmentation.

In conclusion we may state that this method seems at-

```
THE RESULT FOR OBJECT    1


  THE ORIGINAL CORNER LOCATIONS(LEVEL 0)

      1. ( 25, 31)  2. ( 25, 23)  3. ( 16, 20)  4. ( 26, 18)
      5. ( 30,  9)  6. ( 32, 17)  7. ( 47,  8)  8. ( 39, 17)
      9. ( 50, 15) 10. ( 54, 18) 11. ( 36, 21) 12. ( 36, 25)
     13. ( 30, 22) 14. ( 28, 25) 15. ( 31, 30) 16. ( 47, 22)
     17. ( 41, 30) 18. ( 55, 30) 19. ( 55, 26) 20. ( 61, 31)
     21. ( 56, 35) 22. ( 39, 35) 23. ( 48, 46) 24. ( 47, 55)
     25. ( 39, 42) 26. ( 30, 35) 27. ( 24, 42) 28. ( 23, 35)
     29. ( 12, 34)

LEVEL   1

  A CONVEX SUBPART BETWEEN   ( 23, 35) ( 25, 31)
  A CONVEX SUBPART BETWEEN   ( 25, 23) ( 26, 18)
  A CONVEX SUBPART BETWEEN   ( 26, 18) ( 32, 17)
  A CONVEX SUBPART BETWEEN   ( 32, 17) ( 39, 17)
  A CONVEX SUBPART BETWEEN   ( 39, 17) ( 36, 21)
  A CONVEX SUBPART BETWEEN   ( 36, 21) ( 30, 22)
  A SPIRAL SUBPART BETWEEN   ( 41, 30) ( 39, 35)
  A CONVEX SUBPART BETWEEN   ( 39, 35) ( 39, 42)
  A CONVEX SUBPART BETWEEN   ( 30, 35) ( 23, 35)
  THE REMAINING CORNERS IN THE CENTRAL SUBPOLYGON
      1. ( 25, 31)  2. ( 25, 23)  3. ( 26, 18)  4. ( 32, 17)
      5. ( 39, 17)  6. ( 36, 21)  7. ( 30, 22)  8. ( 28, 25)
      9. ( 31, 30) 10. ( 47, 22) 11. ( 41, 30) 12. ( 39, 35)
     13. ( 39, 42) 14. ( 30, 35) 15. ( 23, 35)

LEVEL   2

  A CONVEX SUBPART BETWEEN   ( 30, 35) ( 25, 31)
  A SPIRAL SUBPART BETWEEN   ( 25, 31) ( 31, 30)
  A SPIRAL SUBPART BETWEEN   ( 31, 30) ( 39, 35)
  A CONVEX SUBPART BETWEEN   ( 39, 35) ( 30, 35)
  THE REMAINING CORNERS IN THE CENTRAL SUBPOLYGON
      1. ( 25, 31)  2. ( 31, 30)  3. ( 39, 35)  4. ( 30, 35)

LEVEL   3

  A CONVEX SUBPART BETWEEN   ( 25, 31) ( 30, 35)
```

Fig. 20. Computer printout describing the analysis of Fig. 19.

TABLE I
COMPUTATIONAL STATISTICS (IN SECONDS)

Operation \ Subject	Chromosomes			Chinese Characters			
	13	14	15	16	17	18	19
Thresholding	0.60	0.60	0.60	0.67	0.60	0.65	0.69
Polygonal approximation (including bdy ordering)	1.72	2.16	1.48	3.67	3.16	3.22	2.27
Decomposition	0.32	0.45	0.38	0.40	0.40	0.40	0.44
Total	2.64	3.21	2.46	4.74	4.16	4.27	3.40

Note: The above times were printed by the operating system during runs. However their accuracy is only within a factor of ±10 percent.

tractive in applications where registration, variations in size and loosely defined shape are a problem, while it is probably too expensive in applications where the objects are stylized and in fairly well defined positions (e.g., typewritten text). Specific examples of applications will appear in a future paper [50].

REFERENCES

[1] A. C. Shaw, "A formal picture description scheme or a basis for picture processing systems," *Inform. Contr.*, vol. 14, pp. 9–52, 1969.

[2] U. Grenander, "Foundations of pattern analysis," *Quart. Appl. Math.*, vol. 27, pp. 1–55, 1969.

[3] K. S. Fu and P. H. Swain, "On syntactic pattern recognition," in *Software Engineering*, J. Tou, Ed. New York: Academic Press, 1971, pp. 155–182.

[4] T. Pavlidis, "Structural pattern recognition: Primitives and juxtaposition relations," in *Frontiers of Pattern Recognition*, S. Watanable Ed. New York: Academic Press, 1972, pp. 421–451.

[5] K. S. Fu, *Syntactic Methods in Pattern Recognition.* New York: Academic Press, 1974.

[6] L. S. Frishkopf and L. D. Harmon, "Machine reading of cursive script," in *Proc. Symp. Information Theory*, C. Cherry, Ed. London, England: Butterworth, 1961, pp. 300–316.

[7] M. Eden, "Handwriting and pattern recognition," *IRE Trans. Inform. Theory*, vol. IT-8, pp. 160–166, Feb. 1962.

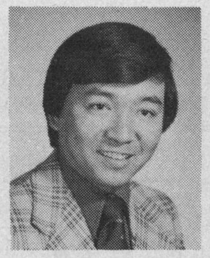
[8] T. Pavlidis, "Computer recognition of figures through decomposition," *Inform. Contr.*, vol. 14, pp. 526–537, May–June 1968.

[9] L. D. Harmon, "Automatic recognition of print and script," *Proc. IEEE*, vol. 60, pp. 1165–1176, Oct. 1972.

[10] S. K. Chang, "An interactive system for Chinese character generation and retrieval," *IEEE Trans. Syst., Man, and Cybern.*, vol. SMC-3, pp. 257–265, May 1973.

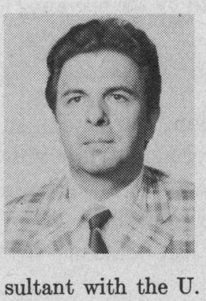
[11] D. L. Caskey and C. L. Coates, "Machine recognition of hand-

printed characters," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 41–49.
[12] Y. Nakano, K. Nakata, Y. Uchikura, and A. Nakajima, "Improvement of Chinese character recognition using projection profiles," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 172–178.
[13] S. Yamamoto, A. Nakajima, and K. Nakata, "Chinese character recognition by hierarchical pattern matching," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 187–196.
[14] M. Yoshida and M. Eden, "Handwritten Chinese character recognition by an analysis-by-synthesis method," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 197–204.
[15] R. S. Ledley, "High speed automatic analysis of biomedical pictures," *Science*, vol. 146, pp. 216–223, Oct. 1964.
[16] R. S. Ledley *et al.*, "FIDAC: Film input to digital automatic computer and associated syntax-directed pattern-recognition programming system," in *Optical and Electro-Optical Information Processing*, J. T. Tippett *et al.*, Ed. Cambridge, Mass.: Mass. Inst. Tech. Press, 1965, pp. 591–613.
[17] G. Gallus, "Contour analysis in pattern recognition for human chromosome classification," *Applicazioni Bio-Mediche del Calcolo Elettronico*, pp. 95–108, 1968.
[18] D. Rutovitz, "Centromere finding: Some shape descriptors for small chromosome outlines," *Machine Intelligence*, vol. 5, pp. 435–462, 1970.
[19] A. Klinger, A. Kochman, and N. Alexandridis, "Computer analysis of chromosome patterns: Feature-encoding for flexible decisions mating," *IEEE Trans. Comput.*, vol. C-20, pp. 1014–1022, Sept. 1971.
[20] R. S. Ledley, "Some clinical applications of pattern recognition," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 89–112.
[21] M. Aiello, C. Lami, and U. Montanari, "A system for computer measurements and karyotyping of wheat metaphases," in *Prof. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 205–219.
[22] A. Rosenfeld, *Picture Processing by Computer*. New York: Academic Press, 1969.
[23] R. O. Duda and P. E. Hart, *Pattern Classification and Scene Analysis*. New York: Wiley, 1973.
[24] A. Rosenfeld and E. Johnston, "Angle defection on digital curves," *IEEE Trans. Comput.*, vol. C-22, pp. 874–878, Sept. 1973.
[25] T. Pavlidis and S. L. Horowitz, "Piecewise approximation of plane curves," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, pp. 396–405, Oct. 1973.
[26] T. Pavlidis, "Segmentation of pictures and maps through functional approximation," *Comput. Graph. and Image Processing*, vol. 1, pp. 360–372, 1972.
[27] H. Y. Feng and T. Pavlidis, "Finding "vertices" in a picture," *Comput. Graph. and Image Processing*, vol. 2, pp. 103–117, Oct. 1973.
[28] ——, "The generation of polygonal outlines of objects from grey level pictures," *IEEE Trans. Circuits Syst.*, to be published.
[29] T. Pavlidis, "Analysis of set patterns," *Pattern Recognition*, vol. 1, pp. 165–178, Nov. 1968.
[30] ——, "Representation of figures by labeled graphs," *Pattern Recognition*, vol. 4, pp. 5–17, Jan. 1972.
[31] J. E. Hopcroft, and J. D. Ullman, *Formal Languages and Their Relation to Automata*. Reading, Mass.: Addison-Wesley, 1969.
[32] H. Y. Feng, "Description and recognition of scenes and object shapes from gray level inputs," Ph.D. dissertation, Dep. Elec. Eng., Princeton Univ, Princeton, N. J., Aug. 1974.
[33] C. B. Shelman and D. Hodges, "A general purpose program for the extraction of physical features," in *Symp. Feature Extraction and Selection in Pattern Recognition*. Argonne Nat. Lab., pp. 135–144, Oct. 1970.
[34] D. J. Langridge, "On the computation of shape," *Frontiers of Pattern Recognition*, S. Watanable, Ed. New York: Academic Press, 1972.
[35] K. Hanakata, "Feature description for compact pattern description for compact pattern description," in *Prof. 1st Int. Joint Conf. Pattern Recognition*, Washington, D. C., Oct. 1973, pp. 416–422.
[36] D. E. McClure, "Problems and methods of nonlinear feature generation in pattern analysis," presented at the *1974 Princeton Conf. Information Sciences and Systems*, Princeton, N. J.
[37] F. A. Valentine, *Convex Sets*. New York: McGraw-Hill, 1964, p. 157.
[38] M. B. Clowes, "Pictorial relationships—A syntactic approach," *Machine Intelligence*, vol. 4, pp. 361–383, 1969.
[39] J. Mylopoulos and T. Pavlidis, "On the topological properties of quantized spaces: 2. Connectivity and order of connectivity," *J. Ass. Comput. Mach.*, vol. 18, pp. 247–254, Apr. 1971.
[40] B. W. Arden, personal communication.
[41] H. Blum, "A transformation for extracting new descriptors of shape," in *Symp. on Models for the Perception of Speech and Visual Form.* Cambridge, Mass.: Mass. Inst. Tech. Press, 1964.
[42] L. A. Zadeh, "Fuzzy sets," *Inform. Contr.*, vol. 8, pp. 338–353, 1965.
[43] H. J. Bremermann, "Cybernetic functionals and fuzzy sets," in *1971 IEEE Systems, Man, and Cybernetics Annu. Symp. Rec.*, pp. 248–253.
[44] A. Rosenfeld and J. L. Pfaltz, "Web grammars," in *Proc. Joint Int. Conf. Artificial Intelligence*, Washington, D. C., 1969.
[45] U. Montanari, "Heuristically guided search and chromosome matching," *Artificial Intelligence*, vol. 1, pp. 227–245, 1970.
[46] T. Pavlidis, "Linear and context-free graph grammars," *J. Ass. Comput. Mach.*, vol. 19, pp. 11–22, Jan. 1972.
[47] T. Huang and K. S. Fu, "Stochastic syntactic analysis for programmed grammars and syntactic pattern recognition," *Comput. Graph. and Image Processing*, vol. 2, pp. 257–283, 1972.
[48] B. Moayer and K. S. Fu, "A syntactic approach to fingerprint pattern recognition," in *Proc. 1st Int. Joint Conf. Pattern Recognition*, pp. 423–432, Oct. 1973.
[49] K. S. Fu and B. K. Bhargava, "Tree-systems for syntactic pattern recognition," *IEEE Trans. Comput.*, vol. C-22, pp. 1087–1099, Dec. 1973.
[50] L. P. Chang and T. Pavlidis, in preparation.



Hou-Yuan F. Feng was born in Shanghai, China, on February 25, 1947. He received the B.S. degree from National Taiwan University, Taipei, Taiwan, in 1969 and Ph.D. degree from Princeton University, Princeton, N. J., in 1974, both in electrical engineering.

He is currently employed at Pattern Analysis and Recognition Corporation, Rome, N. Y. His current research interests are image processing, picture description, data compaction, and software reliability.



Theodosios Pavlidis (S'62–M'64) was born in Salonica, Greece, on September 8, 1934. He received the diploma from the National Technical University of Athens, Athens, Greece, in 1957, and the M.S. and Ph.D. degrees from the University of California, Berkeley, in 1962 and 1964, respectively.

Since 1964 he has been with the Department of Electrical Engineering, Princeton University, Princeton, N.J., and he is presently an Associate Professor. He is a consultant with the U. S. Army (Frankford Arsenal). His research interests include pattern recognition, image processing, approximation theory, and the application of mathematical techniques in the life sciences. He is the author of *Biological Oscillators: Their Mathematical Analysis* (New York: Academic, 1973).

Dr. Pavlidis is a member of the Association for Computing Machinery, the Pattern Recognition Society, and Sigma Xi.